

Machine Learning Interview Questions

Basics & Key Equations

*Covering: Linear & Logistic Regression · Gradient Descent & Backprop
Decision Trees · Random Forests · SVM · KNN · Naive Bayes
Gradient Boosting · Neural Networks · CNNs · RNNs/LSTMs
Transformers · K-Means · PCA · Model Evaluation · Statistics*

How to use this guide:

Each question is followed by a concise answer.
Key equations are highlighted in purple boxes.
Focus on understanding the *intuition*, not memorising.

Contents

1	Linear Regression	2
2	Logistic Regression	3
3	Gradient Descent & Optimisation	4
4	Neural Networks & Backpropagation	5
5	Decision Trees & Ensemble Methods	6
6	Support Vector Machines (SVM)	7
7	K-Nearest Neighbours (KNN)	8
8	Naive Bayes	9
9	Convolutional Neural Networks (CNNs)	9
10	Recurrent Networks & LSTMs	10
11	Transformers & Attention	11
12	Unsupervised Learning: K-Means & PCA	12
13	Model Evaluation & Selection	13
14	Statistical Tests (Basics)	14
15	Quick-Fire Common Questions	15
15.1	Overfitting vs. Underfitting	15
15.2	Feature Scaling	16
15.3	Handling Imbalanced Data	16
15.4	Missing Data	16
15.5	The Elbow Method	17

1 Linear Regression

Q 1.1

What is linear regression and when would you use it?

Answer: Linear regression models a continuous target y as a weighted sum of input features $\mathbf{x} \in \mathbb{R}^d$:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b$$

Use it when you expect a *linear* relationship between inputs and a continuous output (e.g. predicting house prices, forecasting sales).

Q 1.2

What is the cost function for linear regression?

Answer: Mean Squared Error (MSE):

Cost Function — MSE

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

We minimise this with respect to $\mathbf{w}$ to find the best-fit parameters.

Q 1.3

What is the closed-form solution (Normal Equation)?

Answer: Setting $\nabla_{\mathbf{w}} \mathcal{L} = 0$ and solving analytically:

Normal Equation

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

This is exact but costs $\mathcal{O}(d^3)$ to invert $(\mathbf{X}^\top \mathbf{X})$. For large d , use gradient descent instead.

Q 1.4

What assumptions does linear regression make?

Answer: The **LINE** assumptions:

- **Linearity:** $\mathbb{E}[y|\mathbf{x}]$ is linear in $\mathbf{x}$.
- **Independence:** residuals ε_i are independent.
- **Normality:** residuals $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$.
- **Equal variance:** homoscedasticity ($\text{Var}[\varepsilon_i] = \sigma^2$).

Violations require remedies: transformations, robust regression, GLS, etc.

Q 1.5

What is R^2 and what does it measure?

Answer: R^2 (coefficient of determination) measures the proportion of variance in y explained by the model:

R^2

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \in [0, 1]$$

$R^2 = 1$: perfect fit. $R^2 = 0$: model is no better than predicting the mean.

Q 1.6

What is the difference between Ridge and Lasso regression?

Answer: Both add a regularisation penalty to prevent overfitting:

- **Ridge (L2):** $\mathcal{L} + \lambda \|\mathbf{w}\|_2^2$ — shrinks all coefficients toward zero, but rarely to exactly zero.
- **Lasso (L1):** $\mathcal{L} + \lambda \|\mathbf{w}\|_1$ — drives some coefficients to *exactly* zero (feature selection).
- **ElasticNet:** linear combination of L1 and L2.

A common follow-up: “When would you prefer Lasso over Ridge?”

Answer: when you suspect many features are irrelevant (Lasso gives a sparse solution and acts as automatic feature selection).

2 Logistic Regression

Q 2.1

What is logistic regression? Is it a classifier or a regressor?

Answer: Despite the name, logistic regression is a **classifier**. It models the probability that a sample belongs to class 1 using the sigmoid function:

Sigmoid / Logistic Function

$$\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}} \in (0, 1)$$

Predict class 1 if $\hat{p} \geq 0.5$, class 0 otherwise.

Q 2.2

What is the loss function for logistic regression and why not MSE?

Answer: The **Binary Cross-Entropy (Log Loss)**:

Binary Cross-Entropy Loss

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

MSE with a sigmoid output creates a non-convex loss with many local minima. Cross-entropy is convex and has nice gradient properties.

Q 2.3

What is the decision boundary in logistic regression?

Answer: The decision boundary is where $\hat{p} = 0.5$, i.e. where $\mathbf{w}^\top \mathbf{x} = 0$. This is a **linear hyperplane** in feature space. Logistic regression cannot model non-linear boundaries unless features are engineered (polynomial, interaction terms).

3 Gradient Descent & Optimisation

Q 3.1

What is gradient descent?

Answer: Gradient descent is an iterative optimisation algorithm that minimises a loss function $\mathcal{L}(\mathbf{w})$ by repeatedly moving in the direction of steepest descent (negative gradient):

Gradient Descent Update Rule

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

where $\eta > 0$ is the **learning rate**.

Q 3.2

What is the difference between Batch GD, SGD, and Mini-Batch GD?

Answer:

- **Batch GD:** compute gradient on the *entire* dataset each step. Exact gradient; slow and memory-intensive.
- **SGD (Stochastic):** use a *single* random sample per step. Fast and noisy; can escape local minima.
- **Mini-Batch GD:** use a small batch ($B = 32$ – 256). Best trade-off; standard in deep learning.

Modern optimisers: **Adam** = SGD + momentum + adaptive learning rate.

Q 3.3

What is the learning rate and why does it matter?

Answer: The learning rate η controls the step size. Too large: oscillates or diverges. Too small: converges too slowly or gets stuck. Common strategies: learning rate schedules (cosine annealing), warm-up, or adaptive rates (Adam, AdaGrad, RMSProp).

4 Neural Networks & Backpropagation

Q 4.1

What is a neural network?

Answer: A neural network is a stack of **layers** where each layer applies a linear transformation followed by a non-linear **activation function**:

$$\mathbf{a}^{(l)} = f\left(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right)$$

By composing many such layers, the network can approximate arbitrarily complex functions (Universal Approximation Theorem).

Q 4.2

What is backpropagation? Write the key equation.

Answer: Backpropagation is the algorithm that efficiently computes gradients of the loss w.r.t. all parameters using the **chain rule**. Define the **error signal** at layer l as:

Backpropagation — Chain Rule

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \left(W^{(l+1)\top} \delta^{(l+1)}\right) \odot f'(\mathbf{z}^{(l)})$$

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \mathbf{a}^{(l-1)\top} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

where $\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ is the pre-activation, f' its elementwise derivative, and $\odot$ is elementwise multiplication.

Forward pass: compute all $\mathbf{a}^{(l)}$ and $\mathbf{z}^{(l)}$.

Backward pass: propagate δ from output to input, accumulating gradients.

Q 4.3

What are common activation functions?

Answer:

Function	Formula	Common use
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	Output layer (binary)
Tanh	$\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$	Hidden layers (RNNs)
ReLU	$\max(0, z)$	Hidden layers (CNNs, MLPs)
Leaky ReLU	$\max(0.01z, z)$	Fixes dying ReLU
Softmax	$e^{z_k} / \sum_j e^{z_j}$	Output layer (multi-class)

Q 4.4

What is the vanishing gradient problem?

Answer: In deep networks, gradients are multiplied together through many layers during backprop. If activations have derivatives < 1 (like sigmoid: $\sigma'(z) \leq 0.25$), gradients *shrink exponentially* as they propagate backward, making early layers learn very slowly. **Fixes:** ReLU activations, batch normalisation, residual connections (ResNets), careful weight initialisation (He, Xavier).

Q 4.5

What is dropout and why is it used?

Answer: Dropout randomly sets a fraction p of neuron activations to zero during training. This prevents neurons from co-adapting and acts as a **regulariser** — implicitly training an ensemble of 2^N sub-networks. At inference, all neurons are active but scaled by $1 - p$.

Q 4.6

What is batch normalisation?

Answer: BatchNorm normalises the activations within each mini-batch to have zero mean and unit variance, then applies learnable scale and shift:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \quad y_i = \gamma \hat{z}_i + \beta$$

Benefits: faster convergence, allows higher learning rates, reduces sensitivity to initialisation.

5 Decision Trees & Ensemble Methods

Q 5.1

How does a decision tree split nodes? What metrics does it use?

Answer: At each node, a feature and threshold are chosen to maximise the *purity* of the resulting child nodes. Two common metrics:

Entropy & Information Gain

$$H(S) = - \sum_k p_k \log_2 p_k \quad IG = H(S) - \sum_{\text{branch}} \frac{|S_b|}{|S|} H(S_b)$$

Gini Impurity

$$G(S) = 1 - \sum_k p_k^2$$

where p_k is the proportion of class k in node S . The split that maximises IG (or minimises Gini) is chosen.

Q 5.2

What is overfitting in decision trees? How do you prevent it?

Answer: An unconstrained tree will grow until each leaf has a single sample (perfect training accuracy, poor generalisation). **Prevention:**

- Limit `max_depth`, `min_samples_leaf`.
- **Pruning:** post-prune by removing branches that don't improve validation performance (cost-complexity pruning).
- Use ensemble methods (Random Forest, boosting).

Q 5.3

What is a Random Forest? How is it different from a single tree?

Answer: Random Forest trains T decision trees on **bootstrap samples** of the data (bagging) and at each split considers only a random subset of features ($\sqrt{d}$ for classification, $d/3$ for regression). Predictions are aggregated by **majority vote** (classification) or **mean** (regression). Key advantages over a single tree: lower variance (averaged predictions), built-in feature importance, Out-of-Bag (OOB) error estimation.

Q 5.4

What is the difference between bagging and boosting?

Answer:

- **Bagging:** train base learners *in parallel* on random bootstrap samples. Reduces **variance**. Example: Random Forest.
- **Boosting:** train base learners *sequentially*, each correcting the errors of the previous. Reduces **bias**. Examples: AdaBoost, Gradient Boosting, XGBoost.

Q 5.5

How does Gradient Boosting work?

Answer: Gradient Boosting builds an additive model:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x})$$

where h_m is a new tree fit to the **negative gradient** (pseudo-residuals) of the loss at step m :

$$r_i^{(m)} = -\frac{\partial \mathcal{L}(y_i, F_{m-1}(\mathbf{x}_i))}{\partial F_{m-1}(\mathbf{x}_i)}$$

For MSE loss, residuals are simply $y_i - \hat{y}_i^{(m-1)}$. **XGBoost** extends this with second-order (Newton) updates and hardware optimisations.

6 Support Vector Machines (SVM)

Q 6.1

What is the intuition behind SVM?

Answer: SVM finds the **maximum-margin hyperplane** separating two classes: the decision boundary that is as far as possible from the nearest points of each class (the **support vectors**). Wider margin $\Rightarrow$ better generalisation.

Q 6.2

What is the SVM optimisation problem?

Answer:

Hard-Margin SVM Primal

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i$$

The margin width is $\frac{2}{\|\mathbf{w}\|}$. For non-separable data, the **soft-margin** SVM adds slack variables $\xi_i \geq 0$ and a penalty $C \sum_i \xi_i$ (C controls the bias-variance trade-off).

Q 6.3

What is the kernel trick in SVM?

Answer: The kernel trick allows SVMs to find non-linear boundaries *without explicitly computing* high-dimensional feature maps $\phi(\mathbf{x})$. It replaces all dot products $\mathbf{x}_i^\top \mathbf{x}_j$ with a kernel function $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$. Common kernels:

- Linear: $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$
- Polynomial: $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^d$
- RBF/Gaussian: $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$

7 K-Nearest Neighbours (KNN)

Q 7.1

How does KNN work?

Answer: KNN is a non-parametric, lazy learner. To classify a query point $\mathbf{x}$: find the K training points nearest to $\mathbf{x}$ (by Euclidean distance) and predict by **majority vote** (classification) or **mean** (regression). No training step — all computation at prediction time.

Q 7.2

How do you choose K ? What happens with very small or very large K ?

Answer:

- $K = 1$: overfits; highly sensitive to noise.
- $K = n$: predicts the majority class for all points (underfitting).
- Optimal K : found by cross-validation. Typical range: 3–20.

Odd values of K avoid ties in binary classification.

Q 7.3

What is the curse of dimensionality and how does it affect KNN?

Answer: In high dimensions, all data points become approximately equidistant from any query point — the “nearest neighbours” are no longer meaningfully close. KNN degrades badly as d increases. **Fix:** reduce dimensionality with PCA before using KNN.

8 Naive Bayes

Q 8.1

State Bayes’ theorem and explain it.

Answer:

Bayes’ Theorem

$$P(C|\mathbf{x}) = \frac{P(\mathbf{x}|C) P(C)}{P(\mathbf{x})}$$

- $P(C)$: prior probability of class C .
- $P(\mathbf{x}|C)$: likelihood of observing $\mathbf{x}$ given class C .
- $P(C|\mathbf{x})$: posterior (what we want to predict).
- $P(\mathbf{x})$: evidence (constant across classes, so we can ignore it).

Q 8.2

What is the “naive” assumption in Naive Bayes?

Answer: Naive Bayes assumes that all features are **conditionally independent** given the class label:

$$P(\mathbf{x}|C) = \prod_{j=1}^d P(x_j|C)$$

This is rarely true in practice, but the algorithm still works surprisingly well for text classification (spam filters, sentiment analysis). The final prediction is:

$$\hat{C} = \arg \max_C P(C) \prod_{j=1}^d P(x_j|C)$$

9 Convolutional Neural Networks (CNNs)

Q 9.1

What is a convolution and why is it used in image processing?

Answer: A convolution slides a small **filter** (kernel) K over the input I and computes dot products at each position:

$$(I * K)[i, j] = \sum_m \sum_n I[i + m, j + n] \cdot K[m, n]$$

This exploits two key properties of images:

- **Local connectivity:** pixels near each other are related (edges, textures are local patterns).
- **Translation invariance:** the same filter detects an edge anywhere in the image (parameter sharing).

Q 9.2

What is pooling and why is it used?

Answer: Pooling (max or average) downsamples feature maps by taking the max/mean over a local window. Benefits:

- Reduces spatial dimensions (fewer parameters downstream).
- Provides some translation invariance.
- Makes features more robust to small shifts/distortions.

Modern architectures sometimes replace pooling with strided convolutions.

Q 9.3

What is transfer learning?

Answer: Transfer learning reuses a model pre-trained on a large dataset (e.g. ImageNet) as a starting point for a new task. The early layers capture general features (edges, textures), while the final layers are **fine-tuned** on the new task. Dramatically reduces data and compute requirements.

10 Recurrent Networks & LSTMs

Q 10.1

What is an RNN and what is it used for?

Answer: A Recurrent Neural Network processes sequences by maintaining a **hidden state** $\mathbf{h}_t$ that is updated at each timestep:

$$\mathbf{h}_t = f(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$$

Used for time-series forecasting, NLP (language modelling), speech recognition — any task where order matters.

Q 10.2

Why do vanilla RNNs suffer from vanishing gradients in sequences?

Answer: During Backpropagation Through Time (BPTT), gradients are multiplied by W_h at each timestep. If $\|W_h\| < 1$, gradients shrink exponentially over long sequences; the model “forgets” early inputs. This is the **long-range dependency problem**.

Q 10.3

How does an LSTM solve this? What are its four gates?

Answer: An LSTM (Long Short-Term Memory) maintains a separate **cell state** $\mathbf{c}_t$ with additive updates (not multiplicative), allowing gradients to flow unchanged over long distances.

LSTM Gate Equations

$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$	Forget gate: what to erase from c
$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$	Input gate: what new info to write
$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$	Candidate cell update
$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$	Cell state update
$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$	Output gate
$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$	Hidden state output

11 Transformers & Attention

Q 11.1

What is the attention mechanism? What is the key equation?

Answer: Attention allows a model to selectively focus on different parts of the input when producing each output. Given queries Q , keys K , and values V :

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The $1/\sqrt{d_k}$ scaling prevents dot products from becoming too large (which would push softmax into saturation).

Q 11.2

What is the difference between self-attention and cross-attention?

Answer:

- **Self-attention:** Q , K , V all come from the *same* sequence. Used in the Transformer encoder/decoder to relate each token to all others.
- **Cross-attention:** Q from one sequence (decoder), K and V from another (encoder output). Used in translation to attend to source tokens while generating target tokens.

Q 11.3

Why are Transformers better than RNNs for NLP tasks?

Answer:

- **Parallelism:** all positions processed simultaneously (RNNs are inherently sequential).
- **Long-range dependencies:** any two tokens interact directly in $\mathcal{O}(1)$ operations (RNNs need $\mathcal{O}(n)$ steps).
- **Scale:** Transformers scale efficiently to billions of parameters (GPT, BERT, T5, LLaMA).

12 Unsupervised Learning: K-Means & PCA

Q 12.1

How does K-Means clustering work?

Answer: K-Means minimises the **Within-Cluster Sum of Squares (WCSS)**:

K-Means Objective

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Lloyd's algorithm:

1. Initialise K centroids (use K-Means++ for best results).
2. **Assign** each point to its nearest centroid.
3. **Update** each centroid to the mean of its assigned points.
4. Repeat 2–3 until convergence.

Converges to a *local* minimum. Always run multiple restarts.

Q 12.2

How do you choose K in K-Means?

Answer: Three principled methods:

- **Elbow method:** plot WCSS vs. K ; pick the “elbow” where the rate of decrease sharply slows.
- **Silhouette score:** measures cohesion vs. separation, $s_i = (b_i - a_i) / \max(a_i, b_i) \in [-1, 1]$. Maximise it.
- **Gap statistic:** compares WCSS to a random baseline.

Q 12.3

What is PCA and what does it compute?

Answer: PCA finds a new orthonormal basis for the data that maximises variance in each successive direction. The principal components are the **eigenvectors** of the sample covariance matrix:

PCA: Eigenvalue Problem

$$\Sigma \mathbf{u}_j = \lambda_j \mathbf{u}_j \quad \Sigma = \frac{1}{n-1} \tilde{X}^\top \tilde{X}$$

Project data to k dimensions: $Z = \tilde{X} Q_k$ where $Q_k = [\mathbf{u}_1, \dots, \mathbf{u}_k]$.

Q 12.4

What is the explained variance ratio in PCA? How do you pick k ?

Answer: The explained variance ratio (EVR) of component j is:

$$\text{EVR}_j = \frac{\lambda_j}{\sum_i \lambda_i}$$

Choose k such that $\sum_{j=1}^k \text{EVR}_j \geq 0.95$ (retain 95% of variance). Visualise with a **scree plot** and look for the “elbow” in the eigenvalue curve.

Common follow-up: “Does PCA always help?”

Answer: Not always. PCA assumes linear structure and discards low-variance directions (which might carry the discriminative signal for classification). For non-linear data, use Kernel PCA, t-SNE, or autoencoders.

13 Model Evaluation & Selection

Q 13.1

What is the bias-variance trade-off?

Answer: For any model, the expected test error decomposes as:

Bias-Variance Decomposition

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}))^2] = \underbrace{\text{Bias}[\hat{f}]^2}_{\text{underfitting}} + \underbrace{\text{Var}[\hat{f}]}_{\text{overfitting}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

Simple models have high bias, low variance. Complex models have low bias, high variance. Regularisation and ensembles manage this trade-off.

Q 13.2

What are precision, recall, and F1-score?

Answer:

Classification Metrics

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$$

Precision: of all predicted positives, how many are correct?

Recall: of all actual positives, how many did we catch?

Use F1 when you need a single metric balancing both.

Q 13.3

When would you use ROC-AUC vs precision-recall?

Answer:

- **ROC-AUC:** good for *balanced* class distributions. Measures overall ranking ability of the model.
- **Precision-Recall AUC:** better for *imbalanced* datasets where the positive class is rare (fraud detection, medical diagnosis). ROC-AUC can be misleadingly optimistic when negatives vastly outnumber positives.

Q 13.4

What is k-fold cross-validation and why do we use it?

Answer: K-fold CV splits the data into k equal folds. Iteratively, one fold is held out as the validation set and the model is trained on the remaining $k - 1$ folds. The final score is the mean across all k folds. **Why:** gives a more robust estimate of generalisation performance than a single train/test split, especially with limited data. Common choice: $k = 5$ or $k = 10$.

Q 13.5

What is data leakage? Give an example.

Answer: Data leakage occurs when information from the test set (or the future) “leaks” into the training process, causing overly optimistic estimates. **Example:** fitting a scaler or PCA on the entire dataset (train + test), then splitting — the test mean/variance was used during training. **Fix:** always fit preprocessing on the *training set only* and transform the test set using those fitted parameters.

14 Statistical Tests (Basics)

Q 14.1

What is a p-value? What is it NOT?

Answer: The p-value is the probability of observing data *at least as extreme* as our sample, **assuming the null hypothesis H_0 is true**. It is **NOT** the probability that H_0 is true. If $p\text{-value} < \alpha$ (typically 0.05), we reject H_0 . A small p-value means the data is unlikely under H_0 , not that H_1 is definitely true.

Q 14.2

What is a t-test? When would you use the one-sample, two-sample, and paired versions?

Answer: All t-tests compare means using a test statistic that follows a Student's *t*-distribution:

One-Sample t-statistic

$$t = \frac{\bar{X} - \mu_0}{s/\sqrt{n}} \sim t_{n-1}$$

- **One-sample:** Is the mean equal to a reference value?
- **Two-sample (Welch's):** Do two independent groups have the same mean?
- **Paired:** Do two related measurements differ? (e.g. Model A vs. B on the *same* test set.)

Q 14.3

What is ANOVA and when do you use it instead of multiple t-tests?

Answer: One-way ANOVA tests whether $k \geq 3$ group means are equal ($H_0 : \mu_1 = \dots = \mu_k$):

F-statistic (ANOVA)

$$F = \frac{SS_{\text{Between}}/(k-1)}{SS_{\text{Within}}/(N-k)} \sim F_{k-1, N-k}$$

Running multiple t-tests inflates the Type I error rate ($P(\text{any false positive}) = 1 - (1 - 0.05)^m$). ANOVA controls this with a single test, then post-hoc tests (Tukey HSD) identify which pairs differ.

Q 14.4

How would you compare two machine learning classifiers statistically?

Answer: If both classifiers are evaluated on the *same* test set, use **McNemar's test** (tests whether the disagreement between classifiers is systematic):

$$\chi^2 = \frac{(n_{01} - n_{10})^2}{n_{01} + n_{10}} \sim \chi_1^2$$

For comparing AUC scores on the same test set, use the **DeLong test**. For comparing models across multiple datasets, use the **Friedman test** followed by the Nemenyi post-hoc test.

15 Quick-Fire Common Questions

15.1 Overfitting vs. Underfitting

Q 15.1

How do you know if your model is overfitting or underfitting?

Answer:

- **Overfitting:** low training error, high test error (high variance). Fix: regularise, get more data, simplify model.
- **Underfitting:** high training *and* test error (high bias). Fix: increase model complexity, add features, reduce regularisation, train longer.

Diagnose with **learning curves**: plot train/val loss vs. training size.

15.2 Feature Scaling

Q 15.2

When do you need to scale features? Which algorithms are sensitive?

Answer: Algorithms that use Euclidean distance or gradient descent are sensitive to feature scale:

- **Scale required:** KNN, SVM, PCA, neural networks, logistic regression, linear regression with GD.
- **Scale NOT required:** Decision trees, random forests, gradient boosting (tree-based; use feature thresholds, not distances).

15.3 Handling Imbalanced Data

Q 15.3

How do you deal with class imbalance?

Answer:

- **Resampling:** oversample minority (SMOTE) or undersample majority.
- **Class weights:** set `class_weight='balanced'` in sklearn — penalises misclassification of minority class more.
- **Threshold tuning:** move classification threshold below 0.5 to increase recall.
- **Metric:** use PR-AUC or F1, not accuracy.

15.4 Missing Data

Q 15.4

How do you handle missing values?

Answer:

- **Deletion:** drop rows/columns if missing is rare and random.
- **Imputation:** mean/median (numerical), mode (categorical).
- **Model-based:** use KNN imputer or iterative imputer (MICE).
- **Indicator feature:** add a binary flag “was this feature missing?” as a new feature (missingness may be informative).
- Tree-based models (XGBoost) handle missing values natively.

15.5 The Elbow Method

Q 15.5

What is the elbow method? Where does it appear in ML?

Answer: The elbow method plots a metric against a hyperparameter and looks for the “knee” where the marginal improvement drops sharply:

- **K-Means:** WCSS vs. K — pick K at the elbow.
- **PCA:** eigenvalue vs. component index (scree plot).
- **Neural Networks:** validation loss vs. epochs (stopping point before overfitting).

Final Note

This guide covers the essentials for an ML interview. For derivations and deeper treatments, refer to the companion document *ml_notes.pdf*. Good luck!